Chapitre 3 — Réseaux virtuels, sécurité et Infrastructure as Code

Module : Cloud Computing, Virtualisation & DevOps — Master Sciences et Techniques

Pr. A. ABASSI

Professeur d'informatique & IA

Laboratoire LAVETE Lab – FST Settat

Faculté des Sciences et Techniques de Settat

Université Hassan 1er

Sommaire

1. Introduction
2. Partie I — Fondamentaux des réseaux
3. Partie II — Réseaux virtuels VirtualBox
4. Partie III — Du réseau virtuel local au Cloud public (VPC)
5. Partie IV — Routage
6. Partie V — Ports et services
7. Partie VI — Administration SSH
8. Partie VII — Firewall
9. Partie VIII — UFW en pratique
10. Partie IX — Séquence critique
11. Partie X — Firewall dans le Cloud
12. Partie XI — Utilisateurs & groupes Linux
13. Partie XII — Moindre privilège
14. Partie XIII — IAM dans le Cloud
15. Partie XIV — Infrastructure as Code
16. Partie XV — Vagrant
17. Partie XVI — Le Vagrantfile
18. Partie XVII — Débogage du Vagrantfile
19. Partie XVIII — Terraform et outils IaC
20. Partie XIX — Étude de cas intégratrice
21. Synthèse
22. Corrigés des mini-exercices
23. Mini-quiz
24. TP guidé

1. Introduction

Objectifs d'apprentissage

À l'issue de ce chapitre, l'étudiant sera capable de :

1. **Concevoir** un plan d'adressage VLSM et agréger des routes ;
2. **Concevoir** une architecture VPC multi-niveaux (public / privé) et **justifier** ses choix ;
3. **Durcir** un accès SSH (clés, `sshd_config`, fail2ban, bastion) ;
4. **Distinguer** filtrage *stateful* / *stateless* et **analyser** une menace (STRIDE) ;
5. **Lire et écrire** une politique IAM JSON respectant le moindre privilège ;
6. **Automatiser** un déploiement avec Vagrant (provisioning idempotent) et Terraform (`plan`/`apply`) ;
7. **Analyser** un incident de sécurité Cloud réel et proposer des contre-mesures.

Pourquoi ce chapitre ?

- Une VM **isolée** n'a que peu d'intérêt : elle doit **communiquer**.
- Dès qu'une VM communique, elle devient une **cible potentielle** → sécurité indispensable.
- On ne déploie jamais une seule VM "à la main" en Cloud/DevOps → besoin d'**automatisation**.

Les trois piliers du chapitre

1. Réseaux virtuels et adressage
2. Sécurité (SSH, firewall, utilisateurs, IAM)
3. Automatisation (Infrastructure as Code, Vagrant)

Scénario fil rouge

Mission de l'administrateur

Déployer une VM serveur web en :

1. lui attribuant une **adresse IP** dans un réseau virtuel ;
2. l'administrant à distance en **SSH** ;
3. sécurisant son accès avec un **firewall** ;
4. créant des **utilisateurs** à privilèges limités ;
5. automatisant tout le déploiement avec **Vagrant**.

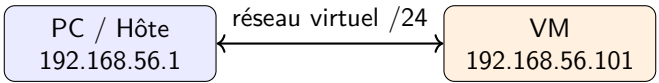
Chaque partie du chapitre répond, une à une, à ce scénario. L'étude de cas finale les réunit toutes.

2. Partie I — Fondamentaux des réseaux

Qu'est-ce qu'un réseau informatique ?

Un **réseau** relie des machines capables d'échanger des données.

- **Machine (hôte)**
- **Interface réseau** (ex. `eth0`)
- **Adresse IP**
- **Réseau / Masque**
- **Passerelle**
- **DNS**
- **Port**
- **Protocole** (TCP, UDP, SSH...)



Adresse IPv4

Une adresse IPv4 : 4 octets (0–255) séparés par des points. Ex. `192.168.56.101`.

Quatre adresses particulières

- **Adresse réseau** : identifie le réseau
- **Adresse hôte** : identifie une machine précise
- **Adresse broadcast** : envoie à toutes les machines du réseau
- **Adresse passerelle** : sortie vers un autre réseau

Cloud

Un VPC AWS est découpé en *subnets*, chacun avec une adresse réseau, une plage utilisable et un broadcast réservé — exactement comme sur VirtualBox.

CIDR et masque réseau : la notation /N

`adresse/N` : `N` bits pour le **réseau**, $32 - N$ bits pour l'**hôte**.

Exemple : 192.168.56.110/24

1. Masque : /24 $\rightarrow$ 255.255.255.0
2. Réseau : 192.168.56.0
3. Broadcast : 192.168.56.255
4. Plage utilisable : 192.168.56.1 à 192.168.56.254
5. Hôtes utilisables : $2^8 - 2 = \mathbf{254}$

Masque	Bits hôte	Adresses	Hôtes utilisables
/24	8	256	254
/25	7	128	126
/26	6	64	62

CIDR : autres exemples calculés

`192.168.56.140/25`

- Blocs de 128 : 0-127 / 128-255
- `.140` → second bloc
- Réseau = `.128`, Broadcast = `.255`
- Plage : `.129` à `.254` (126 hôtes)

`192.168.56.70/26`

- Blocs de 64 : 0-63,64-127...
- `.70` → bloc 64-127
- Réseau = `.64`, Broadcast = `.127`
- Plage : `.65` à `.126` (62 hôtes)

Attention

L'adresse réseau et l'adresse broadcast ne peuvent **jamais** être attribuées à une machine.

Mini-exercice — CIDR

1. Pour `192.168.56.200/26`, déterminer réseau, broadcast, plage utilisable.
2. Combien d'hôtes utilisables offre un `/28` ?
3. `192.168.56.45/25` appartient à quel sous-réseau : `.0/25` ou `.128/25` ?

(corrigé en fin de chapitre)

Adresses IP privées

Plage privée	Taille
`10.0.0.0/8`	∼16,7 M
`172.16.0.0/12`	∼1 M
`192.168.0.0/16`	∼65 000

`192.168.56.101` $\in$ `192.168.0.0/16` $\rightarrow$ **adresse privée**.

IP privée

- Valable en local uniquement
- Réutilisable ailleurs
- Gratuite

IP publique

- Unique au monde
- Joignable depuis Internet
- Attribuée par un fournisseur

VLSM : adapter le masque au besoin

VLSM (*Variable Length Subnet Masking*) : découper un réseau en sous-réseaux de **tailles différentes**.

Méthode en 4 étapes

1. Lister les besoins en hôtes de chaque sous-réseau ;
2. Trier par taille **décroissante** ;
3. Choisir le plus petit préfixe tel que $2^h - 2 \geq$ besoin ;
4. Allouer les blocs **à la suite**, chacun aligné sur une frontière de sa taille.

Pourquoi décroissant ?

Les grands blocs doivent être alignés en premier : sinon la fragmentation rend impossible l'allocation des suivants.

VLSM : exemple de plan d'adressage

Réseau disponible : `192.168.10.0/24`. Besoins : Web 100, App 50, DB 20, Admin 10 hôtes.

Sous-réseau	Besoin	Masque	Adresse réseau	Plage utilisable	Broadcast
Web	100	/25	`192.168.10.0`	`.1` – `.126`	`.127`
App	50	/26	`192.168.10.128`	`.129` – `.190`	`.191`
DB	20	/27	`192.168.10.192`	`.193` – `.222`	`.223`
Admin	10	/28	`192.168.10.224`	`.225` – `.238`	`.239`

Reste libre : `192.168.10.240/28` (réserve pour évolution).

Cloud

AWS réserve **5 adresses** par sous-réseau (réseau, routeur, DNS, usage futur, broadcast) : un /28 n'offre que $16 - 5 = 11$ adresses utilisables.

Exercice)

Exercice

À partir de `10.20.0.0/22`, concevoir un plan VLSM pour : Web 200, App 100, DB 50, Mgmt 20 hôtes. Donner réseau, masque, plage et broadcast de chacun, puis indiquer l'espace restant.

3. Partie II — Réseaux virtuels VirtualBox

Mode NAT

- VM $\rightarrow$ Internet : **oui**, VirtualBox traduit l'adresse
- Hôte $\rightarrow$ VM : **non** (sauf redirection de port)
- Avantages : simple, Internet immédiat
- Limites : VM isolée des autres VM, non administrable depuis l'hôte

Mode Host-only

- VM ↔ Hôte : **oui**
- VM ↔ VM : **oui** (même sous-réseau virtuel)
- Internet : **non** par défaut
- Usage pédagogique : mode recommandé en TP (IP fixe, SSH stable)

Exemple

`192.168.56.101` et `192.168.56.102` partagent le même sous-réseau `192.168.56.0/24` → elles communiquent directement, sans routeur.

Internal Network & Bridged Adapter

Internal Network

Réseau isolé entre VM, **sans aucune interface avec l'hôte** (contrairement à Host-only). Simule un réseau fermé.

Bridged Adapter

La VM se connecte directement à la carte réseau physique de l'hôte $\rightarrow$ obtient une IP sur le **même réseau physique** (ex. Wi-Fi de la salle), comme un poste réel.

Tableau comparatif des modes VirtualBox

Mode	VM↔ VM	VM↔ Hôte	Internet	Réseau physique
NAT	Non	Non	Oui	Non
Host-only	Oui	Oui	Non	Non
Internal Network	Oui	Non	Non	Non
Bridged	Oui	Oui	Oui	Oui

À retenir

En TP : **NAT** (Internet) + **Host-only** (administration SSH) combinés sur la même VM.

Mini-exercice — Modes réseau

1. 20 VM d'étudiants doivent communiquer entre elles, sans Internet ni visibilité depuis le réseau campus. Quel mode ?
2. Une VM doit être testée comme un vrai poste connecté au Wi-Fi de l'université. Quel mode ?

4. Partie III — Du réseau virtuel local au Cloud public (VPC)

Le concept central : VPC

VPC (*Virtual Private Cloud*, ou *VNet* chez Azure) : réseau privé virtuel appartenant à un client Cloud, isolé des autres.

On y retrouve : réseau virtuel, sous-réseaux, IP privées, table de routage, règles de firewall (*Security Groups*).

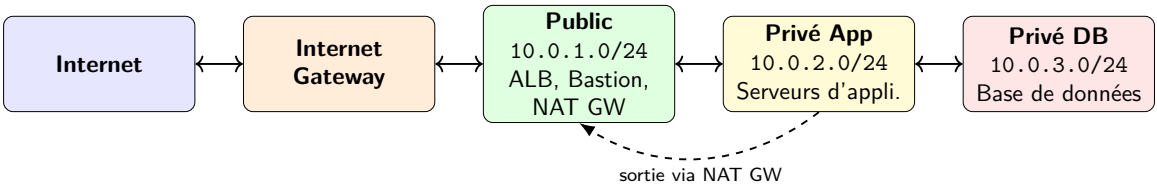
Analogie VirtualBox → Cloud public

VirtualBox	**Cloud public**
NAT	Passerelle Internet + sous-réseau public
Host-only / Internal	Sous-réseau privé
Bridged	Instance sur le réseau physique du datacenter
Réseau virtuel	VPC / VNet
Règles firewall (UFW)	Security Groups
Utilisateurs Linux	IAM

Attention

Analogie **fonctionnelle**, pas une équivalence technique parfaite (un Security Group est *stateful* et agit hors de l'OS de la VM).

Architecture VPC à trois niveaux



VPC `10.0.0.0/16`. Seule la couche publique est exposée ; chaque niveau n'accepte que le trafic du niveau précédent (Security Groups chaînés).

Tables de routage : ce qui rend un sous-réseau ń public ż

Sous-réseau	Destination	Cible
Public	`10.0.0.0/16`	local
	`0.0.0.0/0`	Internet Gateway
Privé App	`10.0.0.0/16`	local
	`0.0.0.0/0`	NAT Gateway
Privé DB	`10.0.0.0/16`	local *(aucune sortie)*

À retenir

Un sous-réseau est **public** s'il possède une route vers l'Internet Gateway (et que ses instances ont une IP publique). Le NAT Gateway permet la **sortie** sans autoriser l'**entrée** depuis Internet.

Composants d'un VPC

Composant	Rôle
Internet Gateway	Point d'entrée/sortie entre le VPC et Internet
NAT Gateway	Sortie Internet des sous-réseaux privés (mises à jour), sans entrée
Bastion	Unique point d'accès SSH d'administration vers les instances privées
Load balancer	Répartit le trafic vers des instances privées, termine TLS
VPC Peering	Relie deux VPC (relation **non transitive**)
VPN / Direct Connect	Relie le VPC au réseau de l'entreprise (on-premise)

5. Partie IV — Routage

Table de routage

La table de routage indique **par où** envoyer un paquet selon sa destination.

- **Réseau directement connecté** : envoi direct
- **Route par défaut** : tout le reste (souvent $\rightarrow$ Internet)
- **Passerelle** : routeur de sortie

Lire une table de routage

```
ip route
```

```
default via 192.168.56.1 dev enp0s8 proto dhcp
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.101
```

- Ligne 1 : route par défaut → passerelle `192.168.56.1`
- Ligne 2 : réseau directement connecté `192.168.56.0/24`

Mini-exercice

IP `192.168.56.110/24`, passerelle `192.168.56.1`. Quelle route pour joindre `8.8.8.8` ?

6. Partie V — Ports et services

Adresse IP vs Port

- **IP** = la machine (l'immeuble)
- **Port** = le service précis (le numéro d'appartement), 0–65535

Service	Port	Protocole
SSH	22	TCP
HTTP	80	TCP
HTTPS	443	TCP
DNS	53	UDP/TCP

Une même VM héberge plusieurs services grâce aux ports (SSH + HTTP simultanément).

7. Partie VI — Administration SSH

SSH : définition

SSH (*Secure Shell*) : administration distante sécurisée.

- **Chiffrement** : trafic illisible en interception
- **Authentification** : identité prouvée (mot de passe / clé)
- **Intégrité** : données non modifiables sans détection

Connexion SSH

```
ssh utilisateur@adresse_ip
```

- `utilisateur` : compte Linux distant
- `@` : sépare utilisateur et cible
- `adresse_ip` : machine distante

Exemple

```
ssh etudiant@192.168.56.101
```

Port SSH

Défaut = **22**. Autre port :

```
ssh -p 2222 utilisateur@serveur
```

SSH vs Telnet

	SSH	Telnet
Chiffrement	Oui	**Non**
Mot de passe en clair	Non	**Oui**
Recommandé aujourd'hui	Oui	Non (obsolète)

Pourquoi éviter Telnet ?

Toutes les données (y compris mots de passe) circulent en clair sur le réseau.

Diagnostiquer ń Connection refused ż

Causes possibles :

- Service SSH arrêté ou non installé
- Firewall bloque le port 22
- Mauvais port utilisé
- Mauvaise adresse IP

Commandes de diagnostic :

```
systemctl status ssh # service actif ?
ss -tlnp # port 22 en écoute ?
sudo ufw status # firewall bloque-t-il ?
```

Attention

Un firewall qui bloque produit souvent un *timeout*, mais UFW peut aussi renvoyer un refus explicite selon la configuration. Toujours vérifier les deux pistes : service ET firewall.

SSH par clés : principe et génération

- **Clé privée** : reste sur le poste client, protégée par une *passphrase*, **jamais** partagée.
- **Clé publique** : copiée sur le serveur (`~/.ssh/authorized_keys`).
- Le serveur vérifie la possession de la clé privée par un défi cryptographique : **aucun secret ne transite**.

```
# Sur le poste client
ssh-keygen -t ed25519 -C "admin@fst"
# -> ~/.ssh/id_ed25519 (privee) et id_ed25519.pub (publique)
ssh-copy-id -i ~/.ssh/id_ed25519.pub etudiant@192.168.56.130
ssh etudiant@192.168.56.130 # plus de mot de passe
```

Permissions strictes

`chmod 700 ~/.ssh` et `chmod 600 ~/.ssh/authorized_keys` : `sshd` refuse les clés si les droits sont trop ouverts.

Durcir sshd_config

```
# /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers amine sara
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
```

Appliquer sans se verrouiller dehors

```
sudo sshd -t # verifie la syntaxe
sudo systemctl reload ssh # applique sans couper les sessions
# Tester dans un 2e terminal AVANT de fermer la session ouverte
```

Pièges

Ne désactivez le mot de passe qu'**après** avoir testé la clé (même logique que la séquence critique UFW). Vérifier aussi `/etc/ssh/sshd_config.d/` : un fichier peut surcharger vos réglages

fail2ban : bloquer la force brute

fail2ban lit les journaux et bannit (via le firewall) les IP qui échouent trop souvent.

```
# /etc/fail2ban/jail.local
[sshd]
enabled = true
maxretry = 5
findtime = 10m
bantime = 1h
```

```
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd # IP bannies
```

Défense en profondeur côté SSH

Clés uniquement → `AllowUsers` → fail2ban → firewall / Security Group restreint à l'IP d'administration.

Bastion, ProxyJump et tunnels SSH

```
# Atteindre une instance privee via le bastion
ssh -J admin@203.0.113.10 dev01@10.0.2.15

# Tunnel : port local 5432 -> base privee, via le bastion
ssh -L 5432:10.0.3.20:5432 admin@203.0.113.10
```

```
# ~/.ssh/config
Host bastion
HostName 203.0.113.10
User admin
IdentityFile ~/.ssh/id_ed25519
Host app-*
ProxyJump bastion
User dev01
```

Le bastion est le **seul** point exposé en SSH ; il doit être durci, journalisé et surveillé.

8. Partie VII — Firewall

Qu'est-ce qu'un firewall ?

Filtre le trafic **entrant** (incoming) et **sortant** (outgoing) selon des règles.

- **Règle réseau** : autorise/bloque selon port, protocole, source
- **Politique par défaut** : comportement si aucune règle ne correspond

Principe ń deny by default ż

Bloquer **tout** par défaut, puis autoriser **explicitement** uniquement le nécessaire.

UFW (*Uncomplicated Firewall*) : outil de gestion du firewall sous Ubuntu (au-dessus de `netfilter/iptables`).

9. Partie VIII — UFW en pratique

Commandes UFW essentielles (1/2)

```
sudo ufw status
```

État du firewall et règles actives.

```
sudo ufw enable
sudo ufw disable
```

Active / désactive le firewall.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

Politiques par défaut : tout entrant bloqué, tout sortant autorisé.

Commandes UFW essentielles (2/2)

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw deny 8080/tcp
```

Autorise SSH et HTTP entrants ; bloque explicitement le port 8080.

```
sudo ufw status numbered
```

Règles numérotées → permet `ufw delete <numéro>`.

Mini-exercice

Écrire la commande UFW autorisant le port 443/tcp (HTTPS).

10. Partie IX — Séquence critique

Le piège classique

Danger

Un administrateur connecté en SSH active UFW **sans** avoir autorisé le port 22 → politique `deny incoming` coupe immédiatement l'accès SSH → perte d'accès (parfois définitive).

Séquence correcte

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
```

Pourquoi cet ordre ? Et le port 8080 ?

- Les règles peuvent être déclarées dans n'importe quel ordre **tant que UFW n'est pas activé**.
- Ce qui compte : la règle SSH doit être en place **avant** `ufw enable`.
- Bonne pratique : toujours déclarer les autorisations, puis les politiques, puis activer en dernier.

Port 8080

`default deny incoming` bloque déjà tout trafic non autorisé, y compris le 8080. La règle `deny 8080/tcp` explicite est donc redondante techniquement, mais peut être exigée pour **clarifier l'intention** dans un cahier des charges.

11. Partie X — Firewall dans le Cloud

Plusieurs niveaux de firewall

1. **Firewall dans la VM** (UFW) — niveau OS
2. **Security Groups** — attaché à l'instance, niveau plateforme
3. **Network ACL** — niveau sous-réseau entier (AWS)

Niveau	AWS	Azure/GCP
Instance	Security Group	NSG (Azure) / Firewall Rules (GCP)
Sous-réseau	Network ACL	Règles associées au sous-réseau
VM (OS)	UFW/iptables	UFW/iptables

À retenir

UFW protège la VM elle-même ; les règles Cloud protègent **avant** que le trafic n'atteigne la VM. On combine les deux : défense en profondeur.

Stateful vs stateless : Security Group vs NACL

Critère	Security Group	Network ACL
Niveau	Instance (interface)	Sous-réseau
Suivi de connexion	**Stateful**	**Stateless**
Types de règles	Autorisation uniquement	Autorisation **et** refus
Évaluation	Toutes les règles	Par numéro croissant, 1re correspondance
Réponses	Autorisées automatiquement	À autoriser explicitement

Conséquence pratique

Une NACL qui autorise l'entrée sur le port 80 doit aussi autoriser les **ports éphémères** (1024–65535) en sortie, sinon les réponses sont perdues. UFW, lui, est stateful (conntrack).

Modèle de menace : STRIDE appliqué à notre scénario

Menace	Exemple	Contre-mesure
Spoofing	Usurpation d'un admin	Clés SSH, MFA
Tampering	Modification de trafic/config	TLS, intégrité, IaC versionné
Repudiation	ń Ce n'est pas moi ż	`auditd`, CloudTrail
Info disclosure	Fuite de données	Chiffrement, moindre privilège
Denial of service	Saturation du service	NACL/SG, limitation, fail2ban
Elevation of priv.	Utilisateur devenu root	`sudoers` fins, IAM

Zero Trust et responsabilité partagée

Zero Trust

- ń Ne jamais faire confiance, toujours vérifier ż
- Pas de confiance implicite liée à la position réseau
- Authentification + autorisation **à chaque requête**
- Micro-segmentation, moindre privilège

Responsabilité partagée

- **Fournisseur** : datacenter, matériel, hyperviseur
- **Client** : OS invité, configuration réseau, IAM, données
- Un bucket public ou un SG ouvert à `0.0.0.0/0` est **toujours** la faute du client

12. Partie XI — Utilisateurs & groupes Linux

Concepts de base

- **Utilisateur** : compte avec ses propres droits
- **Groupe** : ensemble d'utilisateurs partageant des permissions
- **root** : super-utilisateur, tous les droits
- **sudo** : exécution ponctuelle avec droits root

Créer utilisateurs et groupes

```
sudo adduser amine
sudo adduser sara
sudo adduser youssef
```

```
sudo groupadd admins
sudo groupadd logs
sudo groupadd service-managers
```

Ajouter un utilisateur à un groupe

```
sudo usermod -aG admins amine
```

- `-a` (*append*) : ajoute **sans retirer** des groupes existants
- `-G` : groupe(s) **secondaire(s)**

Attention

Oublier `-a` peut retirer l'utilisateur de groupes importants (ex. `sudo`) sans avertissement !

Mini-exercice

Ajouter `sara` aux groupes `logs` et `service-managers` en une commande.

13. Partie XII — Moindre privilège

Principe du moindre privilège

Définition

Un utilisateur ne doit disposer **que** des privilèges strictement nécessaires à sa tâche.

Amine
Administrateur
→ groupe `admins`

Sara
Journaux + services
→ `logs`, `service-managers`

Youssef
Stagiaire
→ aucun groupe privilégié

Risques du root systématique : suppression accidentelle, modification involontaire, logiciels non autorisés, compromission aggravée en cas de piratage.

14. Partie XIII — IAM dans le Cloud

IAM : Identity and Access Management

- **Utilisateur** : identité pouvant agir sur le Cloud
- **Groupe** : ensemble d'utilisateurs
- **Rôle** : permissions ń endossables ż temporairement
- **Permission** : action précise autorisée/refusée
- **Politique** : document décrivant des permissions

Distinction fondamentale

Authentification : ń Qui es-tu ? ż
Autorisation : ń Qu'as-tu le droit de faire ? ż

Analogie Linux → IAM Cloud

Linux	IAM Cloud
Utilisateur (`adduser`)	Utilisateur IAM
Groupe (`groupadd`)	Groupe IAM
`usermod -aG`	Attachement d'une politique/rôle
Permissions fichiers	Permissions sur ressources Cloud

Exemple AWS

Groupe IAM `Developers` : démarrage/arrêt d'instances autorisé, suppression interdite — comme Sara sous Linux.

Lire une politique IAM (JSON)

```
{
   "Version": "2012-10-17",
   "Statement": [
   { "Effect": "Allow",
       "Action": "ec2:Describe*",
       "Resource": "*" },
   { "Effect": "Allow",
       "Action": ["ec2:StartInstances", "ec2:StopInstances"],
       "Resource": "arn:aws:ec2:eu-west-3:123456789012:instance/*",
       "Condition": { "StringEquals": { "ec2:ResourceTag/Env": "dev" } } },
   { "Effect": "Deny",
       "Action": "ec2:TerminateInstances",
       "Resource": "*" }
   ]
}
```

Effect (Allow/Deny), **Action** (opération), **Resource** (ARN ciblé), **Condition** (contexte requis).

Logique d'évaluation et questions

Ordre d'évaluation

1. Par défaut : **refus implicite**.
2. Un `Allow` explicite applicable autorise l'action.
3. Un `Deny` explicite l'emporte **toujours**.

Questions sur la politique précédente

1. L'utilisateur peut-il démarrer une instance étiquetée `Env=dev` ?
2. Peut-il arrêter une instance étiquetée `Env=prod` ?
3. Peut-il supprimer (`Terminate`) une instance `dev` ?
4. Quel principe est illustré par la condition sur l'étiquette ?

Rôles, STS, MFA, secrets et audit

- **Rôle** : identité sans mot de passe, endossée temporairement (par un utilisateur, un service, une instance EC2).
- **STS** (*Security Token Service*) : délivre des **identifiants temporaires** → limite l'impact d'une fuite.
- **MFA** : second facteur obligatoire pour les comptes à privilèges.
- **Secrets** : jamais dans le code ni dans Git ; utiliser un gestionnaire (Secrets Manager, Vault).
- **Audit** : CloudTrail (qui a fait quoi, quand) côté Cloud ; `auditd` côté Linux.

Bonnes pratiques

Pas de clés d'accès longue durée pour les humains ni pour les applications hébergées sur le Cloud : préférer les rôles. Compte *root* du Cloud : MFA, aucun usage courant.

15. Partie XIV — Infrastructure as Code

Le problème

Question

Pourquoi créer manuellement 20 VM, leur réseau, leurs utilisateurs, un par un ?

Infrastructure as Code (IaC)

Décrire l'infrastructure sous forme de **fichiers reproductibles**, exécutés automatiquement par un outil.

Avantages de l'IaC & concepts clés

Avantages

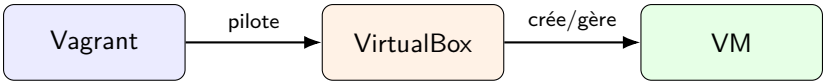
- Automatisation
- Reproductibilité
- Versionnement (Git)
- Réduction des erreurs
- Rapidité
- Environnements identiques
- Suppression/recréation facile

Concepts

- **Configuration** : état souhaité
- **Provisioning** : application de la config
- **Idempotence** : même résultat à chaque exécution
- **Reproductibilité** : environnement identique partout

16. Partie XV — Vagrant

Vagrant : automatiser les VM locales



Vagrant délègue la création réelle à un **provider** (VirtualBox), puis configure automatiquement la VM selon le **Vagrantfile**.

Commandes Vagrant essentielles

```
vagrant init # crée un Vagrantfile de base
vagrant up # crée / démarre la VM
vagrant ssh # connexion SSH automatique
vagrant halt # arrêt propre
vagrant destroy # suppression définitive
vagrant status # état de la VM
```

Attention

`vagrant destroy` est irréversible : les données non partagées avec l'hôte sont perdues.

17. Partie XVI — Le Vagrantfile

Structure d'un Vagrantfile

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/jammy64"
  config.vm.hostname = "vm-web"

  config.vm.network "private_network",
                  ip: "192.168.56.120"

  config.vm.provider "virtualbox" do |vb|
    vb.memory = 2048
    vb.cpus = 2
  end
end
```

Ligne par ligne

- `Vagrant.configure("2")` : version du format de configuration
- `config.vm.box` : image de base (Ubuntu 22.04 / jammy64)
- `config.vm.hostname` : nom de la machine
- `private_network, ip:` : réseau privé + IP fixe
- `provider "virtualbox"` : bloc spécifique VirtualBox
- `vb.memory` / `vb.cpus` : RAM / CPU alloués

Question de vérification

Que modifier pour donner 4 Go de RAM au lieu de 2 ?

18. Partie XVII — Débogage du Vagrantfile

Erreur de syntaxe Ruby fréquente

Incorrect

```
config.vm.network "private_network" ip: "192.168.56.130"
```

Pourquoi ? En Ruby, plusieurs arguments doivent être séparés par une **virgule**. Il en manque une ici.

Correction

```
config.vm.network "private_network",
                  ip: "192.168.56.130"
```

Mini-exercice

Identifier et corriger : `vb.memory = 2048 vb.cpus = 2`

Vagrant : provisioning automatique

Un **provisioner** exécute la configuration **au démarrage** de la VM : plus aucune étape manuelle.

```
Vagrant.configure("2") do |config|
config.vm.box = "ubuntu/jammy64"
config.vm.hostname = "web-01"
config.vm.network "private_network", ip: "192.168.56.130"
config.vm.provision "shell", path: "provision.sh"
end
```

```
#!/bin/bash
set -euo pipefail
export DEBIAN_FRONTEND=noninteractive
apt-get update -y && apt-get install -y nginx ufw
ufw allow 22/tcp && ufw allow 80/tcp # SSH AVANT enable
ufw default deny incoming && ufw default allow outgoing
ufw --force enable
groupadd -f developers
id dev01 &>/dev/null || useradd -m -s /bin/bash -G developers dev01
```

Idempotence du provisioning

Le script doit pouvoir être **rejoué** sans erreur ni effet de bord.

Non idempotent	Idempotent	Pourquoi
`groupadd developers`	`groupadd -f developers`	`-f` : pas d'erreur si existant
`useradd dev01`	`id dev01 || useradd ...`	création conditionnelle
`echo ... >> fichier`	`cp` / modèle de fichier	pas de doublons

```
vagrant up --provision # (re)joue le provisioning au demarrage
vagrant provision # sur une VM deja demarree
vagrant reload --provision # redemarre + provisioning
```

Par défaut, le provisioning ne s'exécute qu'au **premier** `vagrant up`.

19. Partie XVIII — Terraform et outils IaC

Terraform : IaC déclarative pour le Cloud

- On décrit l'**état souhaité** en **HCL** ; Terraform calcule les actions nécessaires.
- Multi-fournisseurs via des **providers** (AWS, Azure, GCP, ...).
- Fork open source compatible : **OpenTofu**.

Workflow

`terraform init` → `fmt` / `validate` → `plan` → `apply` → `destroy`

Élément HCL	Rôle
`provider`	Plugin du fournisseur Cloud
`resource`	Objet à créer et gérer
`variable` / `output`	Entrées / sorties paramétrables
`module`	Bloc réutilisable

Exemple Terraform (1/2) — réseau et pare-feu

```
provider "aws" { region = "eu-west-3" }

variable "admin_cidr" {
    type = string
    description = "IP d'administration autorisee en SSH"
}

resource "aws_vpc" "main" {
    cidr_block = "10.0.0.0/16"
    tags = { Name = "vpc-tp" }
}

resource "aws_subnet" "public" {
    vpc_id = aws_vpc.main.id
    cidr_block = "10.0.1.0/24"
}

resource "aws_security_group" "web" {
    name = "web-sg"
    vpc_id = aws_vpc.main.id
    ingress { from_port = 22 to_port = 22 protocol = "tcp" cidr_blocks = [var.admin_cidr] }
    ingress { from_port = 80 to_port = 80 protocol = "tcp" cidr_blocks = ["0.0.0.0/0"] }
    egress { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }
}
```

Exemple Terraform (2/2) — instance et sortie

```
variable "ami_id" { type = string }

resource "aws_instance" "web" {
    ami = var.ami_id
    instance_type = "t3.micro"
    subnet_id = aws_subnet.public.id
    vpc_security_group_ids = [aws_security_group.web.id]
    tags = { Name = "web-01" }
}

output "web_private_ip" {
    value = aws_instance.web.private_ip
}
```

Simplifications

Internet Gateway, table de routage et IP publique sont omis pour la lisibilité : à ajouter pour un sous-réseau réellement public. `terraform plan` affiche toujours **ce qui va changer** avant `apply`.

État (state), dérive et GitOps

- **State** (`terraform.tfstate`) : correspondance entre le code et les ressources réelles.
- En équipe : **backend distant** (ex. S3) avec **verrouillage**, jamais le state dans Git.
- Le state peut contenir des **secrets** → à chiffrer et à protéger.
- **Dérive (*drift*)** : écart entre l'infrastructure réelle (modifiée à la main) et le code ; détectée par `terraform plan`.
- **GitOps / CI-CD** : le dépôt Git est la source de vérité ; `plan` en *pull request*, `apply` après revue et fusion.

Règle d'or

Toute modification manuelle dans la console Cloud crée de la dérive : modifier le code, pas la console.

Comparatif : Vagrant, Terraform, Ansible, cloud-init

Outil	Paradigme	Périmètre	Usage typique
Vagrant	Vagrantfile déclaratif, scripts	VM locales	TP, dév./test
Terraform	Déclaratif, avec state	Ressources Cloud	Réseau, VM, IAM
Ansible	Déclaratif, idempotent, via SSH	Configuration de machines	Paquets, services, utilisateurs
cloud-init	Script/YAML au 1er démarrage	Instance Cloud	Initialisation de l'instance

Mutable vs immutable

Mutable : on modifie le serveur en place (Ansible). **Immutable** : on remplace le serveur par une nouvelle image (Terraform + images). L'immutable limite la dérive.

À retenir

Outils **complémentaires** : Terraform crée l'infrastructure, Ansible ou cloud-init la configure.

20. Partie XIX — Étude de cas intégratrice

Scénario

VM `web-01` :

- réseau privé, SSH réservé à l'administrateur
- serveur Web, firewall strict
- utilisateur développeur sans droits root
- environnement reproductible avec Vagrant

À déterminer : réseau, IP, ports, règles firewall, utilisateurs, groupes, privilèges, Vagrantfile, déploiement, vérification.

Solution (1/2) — Réseau, ports, firewall

- **Réseau** : Host-only `192.168.56.0/24`
- **IP** : `192.168.56.130`
- **Ports** : 22 (SSH), 80 (HTTP)

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
```

Solution (2/2) — Tout est dans le code

- `Vagrantfile` : box, IP `192.168.56.130`, RAM/CPU, `config.vm.provision "shell"`.
- `provision.sh` : nginx, règles UFW (SSH **avant** `enable`), groupe `developers`, utilisateur `dev01` **sans sudo**.
- Accès : clés SSH uniquement (`PasswordAuthentication no`, `PermitRootLogin no`).

Cohérence avec le message IaC

Aucune commande manuelle dans la VM : `vagrant destroy && vagrant up` redonne **exactement** le même serveur.

Déploiement et vérification

```
# Deploiement
vagrant up

# Verification
vagrant status
vagrant ssh -c "sudo ufw status numbered"
vagrant ssh -c "systemctl status ssh --no-pager"
vagrant ssh -c "ss -tlnp"
vagrant ssh -c "id dev01" # groupe developers, pas de sudo
curl -I http://192.168.56.130 # nginx repond ?

# Reproductibilite
vagrant destroy -f && vagrant up # meme resultat
```

Étude de cas d'incident : de la cause racine aux contre-mesures

Incident	Cause racine	Contre-mesures
Bucket de stockage public	Politique/ACL trop permissive, absence de contrôle	Blocage de l'accès public, revue IAM, alertes de configuration
Clé d'accès exposée dans un dépôt Git public	Secret en clair, clé longue durée	Rôles + STS, scan de secrets (gitleaks), rotation, révocation immédiate
Port SSH ouvert à `0.0.0.0/0`	Security Group mal configuré	Restriction à l'IP admin, bastion, clés, fail2ban

Travail demandé : pour un incident, tracer la chaîne d'attaque, classer les menaces (STRIDE) et proposer les contrôles IAM / réseau / audit qui l'auraient évité.

21. Synthèse

Tableau de synthèse final

Concept	Savoir-faire
IPv4 / CIDR	Calculer réseau, broadcast, hôtes utilisables
VirtualBox	Choisir NAT / Host-only / Internal / Bridged
Routage	Lire une table de routage
SSH	Se connecter et diagnostiquer
UFW	Écrire des règles firewall cohérentes
Utilisateurs/Groupes	Créer et organiser avec le moindre privilège
IAM	Comprendre les permissions Cloud
IaC / Vagrant	Automatiser un déploiement reproductible
Terraform	Situer sa complémentarité avec Vagrant

Correspondance avec le TD

TD	Partie du cours
QCM 1–3	IPv4/privé, Host-only, SSH/port 22
QCM 4–6	VPC, UFW, Firewall+SSH
QCM 7–10	Moindre privilège, IAM, IaC, Vagrant
Ex. 1	IPv4 + VirtualBox + routage
Ex. 2	SSH + diagnostic
Ex. 3	UFW + équivalent Cloud
Ex. 4	Utilisateurs/groupes + moindre privilège
Ex. 5	Vagrant + Vagrantfile + débogage + Terraform
Ex. 6	Synthèse complète (Partie XIX)

22. Corrigés des mini-exercices

Corrigés (1/2) — Réseaux, UFW, utilisateurs, Vagrant

1. `192.168.56.200/26` : réseau `.192`, broadcast `.255`, plage `.193`–`.254` (62 hôtes).
2. `/28` : $2^4 - 2 = 14$ hôtes utilisables.
3. `192.168.56.45/25` appartient à `192.168.56.0/25` (plage 0–127).
4. Route pour `8.8.8.8` : route par défaut `default via 192.168.56.1`.
5. 20 VM sans Internet ni visibilité campus : **Internal Network**. Test comme poste Wi-Fi : **Bridged**.
6. `sudo ufw allow 443/tcp`.
7. `sudo usermod -aG logs,service-managers sara`.
8. 4 Go de RAM : `vb.memory = 4096`.
9. Erreur Ruby : deux instructions sur une ligne sans séparateur ; corriger en deux lignes (`vb.memory = 2048` puis `vb.cpus = 2`) ou avec un point-virgule.

Corrigés (2/2) — VLSM, IAM

Plan VLSM pour 10.20.0.0/22 (0.0 à 3.255)

Sous-réseau	Réseau/masque	Plage utilisable	Broadcast
Web 200	10.20.0.0/24	10.20.0.1–10.20.0.254	10.20.0.255
App 100	10.20.1.0/25	10.20.1.1–10.20.1.126	10.20.1.127
DB 50	10.20.1.128/26	10.20.1.129–10.20.1.190	10.20.1.191
Mgmt 20	10.20.1.192/27	10.20.1.193–10.20.1.222	10.20.1.223

Libre : 10.20.1.224/27 et 10.20.2.0/23.

Questions IAM

1. Oui : `Allow` sur `StartInstances` avec `Env=dev`.
2. Non : la condition n'est pas satisfaite → refus implicite.
3. Non : le `Deny` explicite l'emporte toujours.
4. Moindre privilège (périmètre limité par étiquette).

23. Mini-quiz

Mini-quiz (1/2)

1. Hôtes utilisables dans un /27 ?
2. Deux VM Bridged peuvent-elles obtenir une IP du réseau physique ?
3. Commande affichant la table de routage ?
4. Port SSH par défaut ?
5. Pourquoi éviter Telnet ?

Mini-quiz (2/2)

6. Commande UFW autorisant le port 443/tcp ?
7. Risque d'activer UFW sans autoriser SSH avant ?
8. Rôle de `-a` dans `usermod -aG` ?
9. Différence authentification / autorisation ?
10. Vagrant : usage local ou Cloud production ?

Corrigé du mini-quiz

1. $2^5 - 2 = 30$ hôtes
2. Oui (DHCP du réseau physique)
3. `ip route`
4. Port 22
5. Données (dont mots de passe) transmises en clair
6. `sudo ufw allow 443/tcp`
7. Perte immédiate de l'accès distant
8. Ajoute au groupe sans retirer des groupes existants
9. Authentification = qui ; Autorisation = quels droits
10. Usage local (dév./test) ; Terraform pour le Cloud production

24. TP guidé

TP guidé (30–45 min)

1. Créer une VM : `vagrant init ubuntu/jammy64`
2. Configurer une IP privée dans le Vagrantfile
3. Démarrer et se connecter : `vagrant up` / `vagrant ssh`
4. Vérifier le réseau : `ip a`, `ip route`
5. Vérifier SSH : `systemctl status ssh`
6. Configurer UFW (séquence critique)
7. Créer un utilisateur : `sudo adduser dev01`
8. Créer un groupe : `sudo groupadd developers`
9. Appliquer le moindre privilège : `usermod -aG`
10. Détruire/recréer : `vagrant destroy` puis `vagrant up`

Attention

`sudo ufw enable` et `vagrant destroy` sont des étapes sensibles — vérifier avant d'agir.

25. Conclusion

Résumé du chapitre

- Adressage IPv4/CIDR et modes réseau VirtualBox $\rightarrow$ transposables au Cloud (VPC)
- Administration SSH sécurisée et diagnostic des pannes
- Firewall UFW : *deny by default* + séquence d'activation sûre
- Utilisateurs/groupes Linux organisés selon le moindre privilège $\rightarrow$ IAM Cloud
- Automatisation reproductible avec Vagrant (et complémentarité avec Terraform)

Prochaine étape : mettre en pratique dans le TD & le TP !

Références

- AWS, *Amazon VPC User Guide* et *IAM User Guide* (documentation officielle).
- HashiCorp / OpenTofu, documentation Terraform : providers, state, modules.
- *sshd_config(5)* et *ssh_config(5)*, pages de manuel OpenSSH ; RFC 4253 (SSH Transport Layer).
- RFC 1918 (adresses privées), RFC 4632 (CIDR).
- CIS Benchmarks (Ubuntu Linux, AWS Foundations).
- NIST SP 800-207, *Zero Trust Architecture* ; NIST SP 800-53, *Security and Privacy Controls*.
- Y. Brikman, *Terraform: Up & Running*, O'Reilly.